

Artificial Intelligence

Planning

Christopher Simpkins

Kennesaw State University

1 Classical Planning

Classical planning is the task of finding a sequence of actions to accomplish a goal in a discrete, deterministic, static, fully observable environment. Two previous approaches:

- Graph search, e.g., A^*
- Hybrid propositional logical agent

Two limitations:

- Require ad-hoc heuristics
- Require explicit representation of exponentially large state space.

Planning Domain Definition Language solves these problems using a factored representation based on first-order logic.

- A **state** is a conjunction of ground atomic fluents – single predicates containing no variables.
 - $At(Truck_1, Melbourne)$ is a ground atomic fluent, $At(t_1, from)$ is not.
- PDDL uses **database semantics**, or the **closed-world assumption**: any fluents not mentioned are false, and unique names represent distinct objects.

2 Planning Domain Definition Language (PDDL)

Action schema is a family of ground actions.

- Action name and list of variables
- Precondition: conjunction of literals
 - Action a is **applicable** in state s if $s \models a.precondition$
- Effect: conjunction of literals
 - **Result** of executing action a in state s is s' is applying delete list and add list to s :

- * $DEL(a)$, delete list: remove negative literals in action's effects.
- * $ADD(a)$, add list: add positive literals in action's effects.

Action schema:

$Action(Fly(p, from, to),$

$PRECOND : At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$

Ground (variable-free) action:

$EFFECT : \neg At(p, from) \wedge At(p, to)$

$Action(Fly(P_1, SFO, JFK),$

$PRECOND : At(P_1, SFO) \wedge Plane(P_1) \wedge Airport(SFO) \wedge Airport(JFK)$

$EFFECT : \neg At(P_1, SFO) \wedge At(P_1, JFK)$

3 Air Cargo Transport

$Init(At(C_1, SFO) \wedge At(C_2, JFK) \wedge At(P_1, SFO) \wedge At(P_2, JFK)$

$\wedge Cargo(C_1) \wedge Cargo(C_2) \wedge Plane(P_1) \wedge Plane(P_2)$

$\wedge Airport(JFK) \wedge Airport(SFO))$

$Goal(At(C_1, JFK) \wedge At(C_2, SFO))$

$Action(Load(c, p, a),$

$PRECOND: At(c, a) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$

$EFFECT: \neg At(c, a) \wedge In(c, p))$

$Action(Unload(c, p, a),$

$PRECOND: In(c, p) \wedge At(p, a) \wedge Cargo(c) \wedge Plane(p) \wedge Airport(a)$

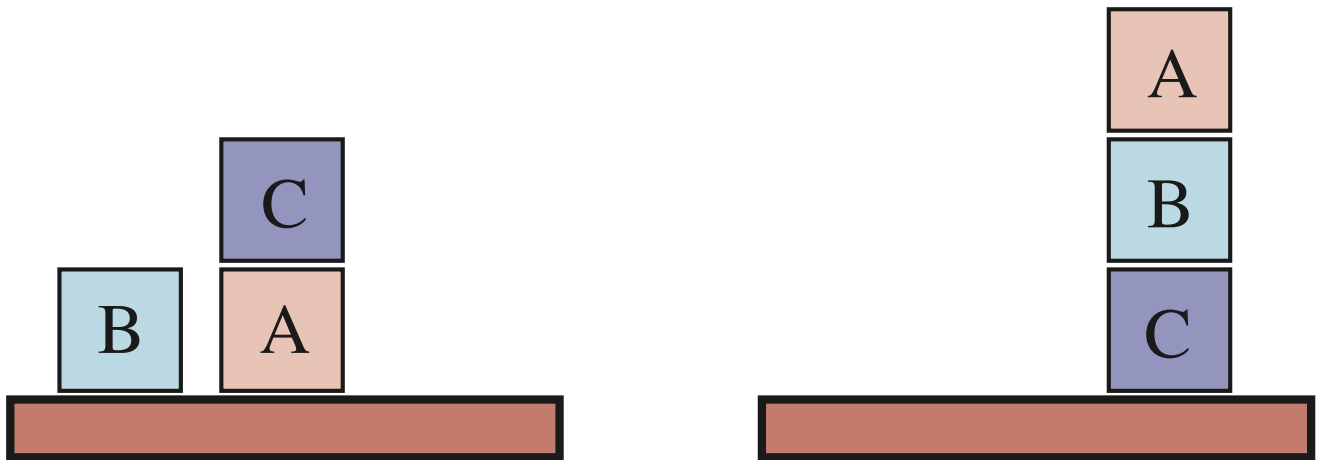
$EFFECT: At(c, a) \wedge \neg In(c, p))$

$Action(Fly(p, from, to),$

$PRECOND: At(p, from) \wedge Plane(p) \wedge Airport(from) \wedge Airport(to)$

$EFFECT: \neg At(p, from) \wedge At(p, to))$

4 Blocks World



Start State

Goal State

5 Blocks World PDDL

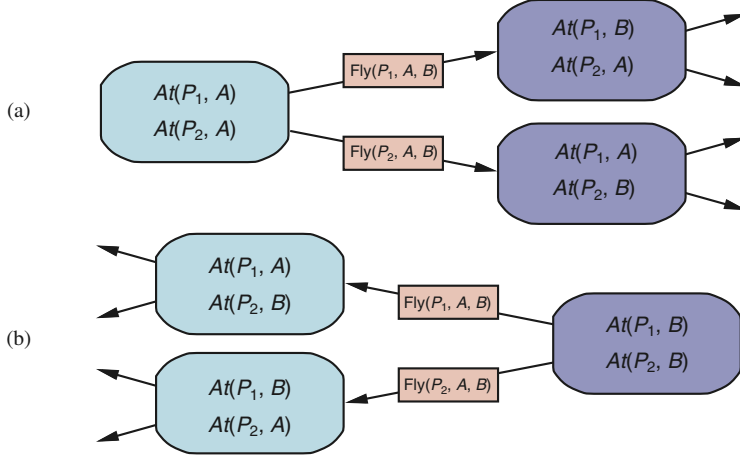
$Init(On(A, Table) \wedge On(B, Table) \wedge On(C, A)$
 $\wedge Block(A) \wedge Block(B) \wedge Block(C) \wedge Clear(B) \wedge Clear(C) \wedge Clear(Table))$
 $Goal(On(A, B) \wedge On(B, C))$
 $Action(Move(b, x, y),$
 PRECOND: $On(b, x) \wedge Clear(b) \wedge Clear(y) \wedge Block(b) \wedge Block(y) \wedge$
 $(b \neq x) \wedge (b \neq y) \wedge (x \neq y),$
 EFFECT: $On(b, y) \wedge Clear(x) \wedge \neg On(b, x) \wedge \neg Clear(y))$
 $Action(MoveToTable(b, x),$
 PRECOND: $On(b, x) \wedge Clear(b) \wedge Block(b) \wedge Block(x),$
 EFFECT: $On(b, Table) \wedge Clear(x) \wedge \neg On(b, x))$

6 Classical Planning Algorithms

- Forward state space search
- Backward state space search
- SATPlan – boolean satisfiability planning
 - Translate PDDL into propositional form, use a SAT solver
- Graphplan
 - Encode constraints related to preconditions and effects in a **planning graph**.
- Situation calculus
- Constraint satisfaction
- Partial-order planning
 - $Remove(Spare, Trunk)$ and $Remove(Flat, Axle)$ must come before $PutOn(Spare, Axle)$, but removals can happen in any order.

7 Forward and Backward State Space Planning

- Forward search: unify current state against preconditions of each action schema – **applicable** actions.
- Backward search: unify goal states against effects of action schemas – **relevant** actions.



8 Hierarchical Planning

Hierarchical task network plans are built from:

- primitive actions, and
- high-level actions (HLA).

HLAs have one or more **refinements**.

- Refinements may contain other HLAs.
- A refinement with only primitive actions is an **implementation**.
- An HLA achieves a goal if at least one of its implementations achieves the goal.

Here are two goal-achieving implementations for the $Go(Home, SFO)$ HLA:

```
Refinement( $Go(Home, SFO)$ ,
  STEPS: [ $Drive(Home, SFO\text{LongTermParking})$ ,
     $Shuttle(SFO\text{LongTermParking}, SFO)$ ] )
```

```
Refinement( $Go(Home, SFO)$ ,
  STEPS: [ $Taxi(Home, SFO)$ ] )
```

Refinements can be produced recursively, as shown in this vacuum world navigation example:

```
Refinement( $Navigate([a, b], [x, y])$ ,
  PRECOND:  $a = x \wedge b = y$ 
  STEPS: [] )
Refinement( $Navigate([a, b], [x, y])$ ,
  PRECOND:  $Connected([a, b], [a - 1, b])$ 
  STEPS: [ $Left, Navigate([a - 1, b], [x, y])$ ] )
Refinement( $Navigate([a, b], [x, y])$ ,
  PRECOND:  $Connected([a, b], [a + 1, b])$ 
  STEPS: [ $Right, Navigate([a + 1, b], [x, y])$ ] )
...
```

9 Closing Thoughts

- Fun to create toy worlds and solve them.
 - Look up "Monkey and bananas" problem.
- Still have knowledge-acquisition bottleneck.
- Still have problem of specifying large number of rules and facts for non-trivial problems.
- Still have problem of uncertainty – nondeterministic actions and partial observability.

In rest of course, we address these issues with uncertain reasoning and machine learning.